```
!pip install yfinance scikit-learn matplotlib
```

```
Requirement already satisfied: yfinance in /usr/local/lib/python3.12/dist-packages (0.2.66)
Requirement already satisfied: scikit-learn in /usr/local/lib/python3.12/dist-packages (1.6.1)
Requirement already satisfied: matplotlib in /usr/local/lib/python3.12/dist-packages (3.10.0)
Requirement already satisfied: pandas>=1.3.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.2.2)
Requirement already satisfied: numpy>=1.16.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.0.2)
Requirement already satisfied: requests>=2.31 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.32.4)
Requirement already satisfied: multitasking>=0.0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.0.12)
Requirement already satisfied: platformdirs>=2.0.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.5.1)
Requirement already satisfied: pytz>=2022.5 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2025.2)
Requirement already satisfied: frozendict>=2.3.4 in /usr/local/lib/python3.12/dist-packages (from yfinance) (2.4.7)
Requirement already satisfied: peewee>=3.16.2 in /usr/local/lib/python3.12/dist-packages (from yfinance) (3.18.3)
Requirement already satisfied: beautifulsoup4>=4.11.1 in /usr/local/lib/python3.12/dist-packages (from yfinance) (4.13.5)
Requirement already satisfied: curl_cffi>=0.7 in /usr/local/lib/python3.12/dist-packages (from yfinance) (0.13.0)
Requirement already satisfied: protobuf>=3.19.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (5.29.5)
Requirement already satisfied: websockets>=13.0 in /usr/local/lib/python3.12/dist-packages (from yfinance) (15.0.1)
Requirement already satisfied: scipy>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.16.3)
Requirement already satisfied: joblib>=1.2.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (1.5.3)
Requirement already satisfied: threadpoolctl>=3.1.0 in /usr/local/lib/python3.12/dist-packages (from scikit-learn) (3.6.0)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.3.3)
Requirement already satisfied: cycler>=0.10 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (0.12.1)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (4.61.1)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (1.4.9)
Requirement already satisfied: packaging>=20.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (25.0)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (11.3.0)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (3.2.5)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib) (2.9.0.post
Requirement already satisfied: soupsieve>1.2 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.11.1->yfinar
Requirement already satisfied: typing-extensions>=4.0.0 in /usr/local/lib/python3.12/dist-packages (from beautifulsoup4>=4.1
Requirement already satisfied: cffi>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance) (2.0.
Requirement already satisfied: certifi>=2024.2.2 in /usr/local/lib/python3.12/dist-packages (from curl_cffi>=0.7->yfinance)
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/python3.12/dist-packages (from pandas>=1.3.0->yfinance) (202
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfi
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance) (3.11
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.31->yfinance)
Requirement already satisfied: pycparser in /usr/local/lib/python3.12/dist-packages (from cffi>=1.12.0->curl_cffi>=0.7->yfir
```

```
import yfinance as yf
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import TimeSeriesSplit
from sklearn.metrics import accuracy_score, classification_report
```

```
STOCK = "CUPID.NS"

end_date = pd.Timestamp.today()
start_date = end_date - pd.DateOffset(years=5)

df = yf.download(STOCK, start=start_date, end=end_date)

df = df[['Close']]
df.columns = ['Close']
df.dropna(inplace=True)

print("Total rows (5 years):", len(df))
```

```
/tmp/ipython-input-2222725482.py:6: FutureWarning: YF.download() has changed argument auto_adjust default to True
  df = yf.download(STOCK, start=start_date, end=end_date)
[*********************100%***********************]  1 of 1 completedTotal rows (5 years): 1237
```

```
# EMAs
df['EMA_20'] = df['Close'].ewm(span=20, adjust=False).mean()
df['EMA_50'] = df['Close'].ewm(span=50, adjust=False).mean()
df['EMA_100'] = df['Close'].ewm(span=100, adjust=False).mean()

# EMA slope
df['EMA_Slope'] = df['EMA_50'] - df['EMA_50'].shift(5)

# RSI
delta = df['Close'].diff()
gain = delta.clip(lower=0).rolling(14).mean()
loss = -delta.clip(upper=0).rolling(14).mean()
rs = gain / loss
df['RSI'] = 100 - (100 / (1 + rs))
```

```
# Momentum
df['Momentum_10'] = df['Close'] - df['Close'].shift(10)
df['Momentum_20'] = df['Close'] - df['Close'].shift(20)

# Volatility
df['Volatility'] = df['Close'].pct_change().rolling(20).std()

# Trend confirmation
df['Trend_Confirm'] = (
    (df['EMA_20'] > df['EMA_50']) &
    (df['EMA_50'] > df['EMA_100'])
).astype(int)
df.dropna(inplace=True)
```

```
df['Future_Return'] = (df['Close'].shift(-20) - df['Close']) / df['Close']

df['Target'] = 0
df.loc[df['Future_Return'] > 0.05, 'Target'] = 1
df.loc[df['Future_Return'] < -0.05, 'Target'] = -1

df.dropna(inplace=True)

print("\nTarget Distribution:")
print(df['Target'].value_counts())
```

```
Target Distribution:
Target
 1    500
 0    423
-1    274
Name: count, dtype: int64
```

```
features = [
    'EMA_20','EMA_50','EMA_100',
    'EMA_Slope','RSI',
    'Momentum_10','Momentum_20',
    'Volatility','Trend_Confirm'
]

X = df[features]
y = df['Target']

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
tscv = TimeSeriesSplit(n_splits=5)
scores = []

for train_idx, test_idx in tscv.split(X_scaled):
    X_train, X_test = X_scaled[train_idx], X_scaled[test_idx]
    y_train, y_test = y.iloc[train_idx], y.iloc[test_idx]

    model = RandomForestClassifier(
        n_estimators=300,
        max_depth=6,
        random_state=42,
        class_weight='balanced'
    )

    model.fit(X_train, y_train)
    preds = model.predict(X_test)
    scores.append(accuracy_score(y_test, preds))

print("\nFold Accuracies:", scores)
print("Mean Accuracy:", round(np.mean(scores), 4))
```

```
Fold Accuracies: [0.592964824120603, 0.2914572864321608, 0.20603015075376885, 0.4020100502512563, 0.21105527638190955]
Mean Accuracy: 0.3407
```

```
model.fit(X_scaled, y)

probs = model.predict_proba(X_scaled)
classes = list(model.classes_)

df['Bull_Prob'] = probs[:, classes.index(1)]
df['Bear_Prob'] = probs[:, classes.index(-1)]

df['Signal'] = 'SIDEWAYS'
```

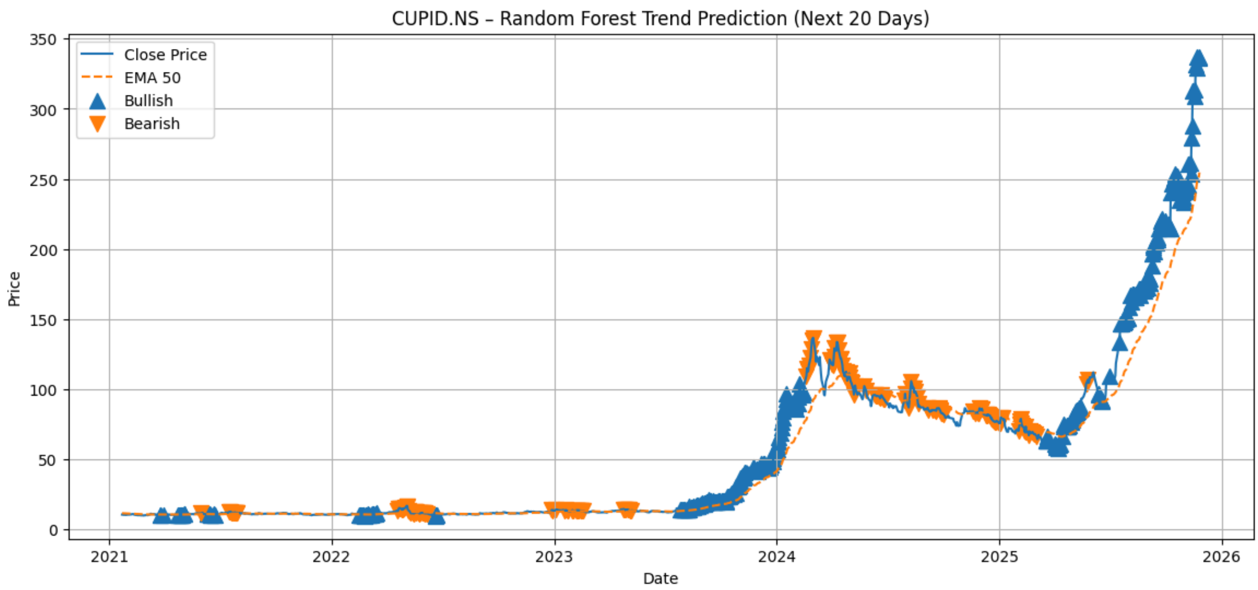
```
df.loc[df['Bull_Prob'] > 0.6, 'Signal'] = 'BULLISH'
df.loc[df['Bear_Prob'] > 0.6, 'Signal'] = 'BEARISH'
```

```
plt.figure(figsize=(14,6))
plt.plot(df.index, df['Close'], label='Close Price')
plt.plot(df.index, df['EMA_50'], linestyle='--', label='EMA 50')

bull = df[df['Signal'] == 'BULLISH']
bear = df[df['Signal'] == 'BEARISH']

plt.scatter(bull.index, bull['Close'], marker='^', s=100, label='Bullish')
plt.scatter(bear.index, bear['Close'], marker='v', s=100, label='Bearish')

plt.title(f"{STOCK} - Random Forest Trend Prediction (Next 20 Days)")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.grid(True)
plt.show()
```




```
print("\nClassification Report (Full Data):")
print(classification_report(y, model.predict(X_scaled)))
```

```
Classification Report (Full Data):
              precision    recall  f1-score   support

          -1       0.77      0.92      0.84       274
           0       0.86      0.82      0.84       423
           1       0.97      0.91      0.94       500

    accuracy                           0.88      1197
   macro avg       0.87      0.88      0.87      1197
weighted avg       0.89      0.88      0.88      1197
```